

```

try:
    import jax
    import jax.numpy as jnp
    import flax.linen as nn
    import optax
    from flax.training import train_state
    from datasets import load_dataset
    from transformers import AutoTokenizer
    import numpy as np
except ImportError as e:
    print(f"ImportError: {e}. Please ensure all required dependencies are installed.")
    raise

# Load and preprocess dataset
def load_and_preprocess_data(batch_size=32, max_length=128):
    dataset = load_dataset("wikitext", "wikitext-2-raw-v1", split="train")
    tokenizer = AutoTokenizer.from_pretrained("gpt2")

    # Set pad token explicitly (GPT-2 does not have one by default)
    tokenizer.pad_token = tokenizer.eos_token

    def tokenize_function(examples):
        """ Tokenize text, ensuring all values are valid numpy arrays. """
        if "text" not in examples:
            return {}

        # Filter out empty strings and None values
        texts = [t for t in examples["text"] if t and t.strip()]
        if not texts:
            return {}

        tokenized = tokenizer(
            texts,
            padding=True, # Pad to longest in batch
            truncation=True,
            max_length=max_length,
            return_tensors="np",
            return_attention_mask=True
        )

        if "input_ids" not in tokenized:
            return {}

        return {
            "input_ids": np.array(tokenized["input_ids"], dtype=np.int32),
            "labels": np.array(tokenized["input_ids"], dtype=np.int32),
        }

    tokenized_datasets = dataset.map(tokenize_function, batched=True)

    def convert_to_jax(batch):
        """ Convert batches safely to JAX arrays, handling variable sizes. """

        # Apply conversion without strict batch size filtering
        tokenized_datasets = tokenized_datasets.map(convert_to_jax, batched=True)

        # Remove empty batches
        tokenized_datasets = tokenized_datasets.filter(lambda x: "input_ids" in x and len(x["input_ids"]) > 0)

        return tokenized_datasets.with_format("numpy"), tokenizer

    def create_train_state(rng, model, learning_rate, input_shape):
        """Creates the training state."""
        params = model.init(rng, jnp.ones(input_shape)) # Initialize model parameters
        tx = optax.adam(learning_rate) # Use Adam optimizer
        return train_state.TrainState.create(apply_fn=model.apply, params=params, tx=tx)

# Train the model
def train_model(epochs=5, batch_size=32):
    try:
        print("Loading and preprocessing data...")
        dataset, tokenizer = load_and_preprocess_data(batch_size=batch_size)

```

```

print(f"Data loaded successfully! Total samples: {len(dataset)}")

rng = jax.random.PRNGKey(0)
model = MiniGPT(vocab_size=tokenizer.vocab_size, dim=256, num_layers=4, num_heads=8)
state = create_train_state(rng, model, 1e-3, (1, 128))

# Training loop
print("Starting training...")
for epoch in range(epochs):
    total_loss = 0
    num_batches = 0

    total_samples = 0
    for batch in dataset:
        if "input_ids" not in batch or len(batch["input_ids"]) == 0:
            print("⚠ Skipping empty batch")
            continue

        current_batch_size = len(batch["input_ids"])
        state, loss = train_step(state, batch)
        total_loss += loss * current_batch_size
        total_samples += current_batch_size

    avg_loss = total_loss / total_samples
    print(f"Epoch {epoch + 1}/{epochs}, Loss: {avg_loss:.4f}")

print("Training complete!")

# Test text generation
prompt = "JAX and Flax are powerful tools for"
generated_text = generate_text(state, tokenizer, prompt)

return {
    "model_initialized": True,
    "final_loss": float(avg_loss),
    "generated_text": generated_text
}
except Exception as e:
    return {"error": str(e)}

class TransformerBlock(nn.Module):
    num_heads: int

    def setup(self):
        self.attention = nn.SelfAttention(num_heads=self.num_heads)
        self.dense = nn.Dense(256)

    def __call__(self, x):
        attn_output = self.attention(x)
        return self.dense(attn_output)

class MiniGPT(nn.Module):
    vocab_size: int
    dim: int
    num_layers: int
    num_heads: int

    def setup(self):
        self.embedding = nn.Embed(self.vocab_size, self.dim)
        self.transformer_blocks = [TransformerBlock(num_heads=self.num_heads) for _ in range(self.num_layers)]

    def __call__(self, input_ids):
        x = self.embedding(input_ids)
        for block in self.transformer_blocks:
            x = block(x)
        return x

# Run training
results = train_model(epochs=5, batch_size=32)
print(results)

```